Introducción a POO - Motor de Juego de Naves en Consola

Laboratorio de Programación (LPR) — 5° 3° B

Autor:

Gabriel Zacarias

Docente a cargo: York Mansilla

Introducción

Este trabajo tiene como objetivo presentar la estructura general de un informe académico siguiendo las normas del estilo APA (7ª edición). Aquí se desarrollarán los contenidos trabajados durante la actividad, los cuales marcan la transición del paradigma estructurado de datos hacia la Programación Orientada a Objetos (POO). Se documentará la implementación individual de un simulador táctico de combate espacial en C + +, analizando los pilares esenciales de la abstracción y el encapsulamiento a través de la creación de clases y la correcta delimitación de accesos en la memoria RAM de la computadora.

Desarrollo

A continuación, se detalla el desarrollo de los conceptos teóricos y prácticos aplicados para el diseño y compilación del motor gráfico interactivo:

Glosario Técnico de Bolsillo (Conceptos de POO)

- **Clase (Class):** Molde técnico o plantilla de tipología avanzada definido por el programador que estructura los atributos y métodos que caracterizarán a una entidad.
- **Objeto (Object):** Instancia física de una clase que el compilador aloja en un espacio real y direccionable de la memoria RAM del sistema operativo.
- **Encapsulamiento:** Principio de ocultamiento de datos que restringe el acceso directo a los atributos de un objeto, protegiéndolos mediante fronteras privadas.
- **Miembros Privados (`private`):** Variables internas de la clase que son accesibles única y exclusivamente por los métodos pertenecientes a esa misma estructura.
- **Miembros Públicos (`public`):** Interfaces y métodos que la clase expone hacia el exterior, permitiendo la comunicación e instanciación desde bloques externos como el `main()`.
- **Constructor:** Método especial que se ejecuta de manera automática al instanciar un objeto. Reserva sus bytes en la memoria RAM y define los estados iniciales de sus atributos.

Estructura Obligatoria del Repositorio (`juegodenaves/`)

El espacio de trabajo individual mantiene la siguiente organización de archivos para el control de versiones en GitHub y GitLab:

```
juegodenaves/
├── .gitignore                  <-- Exclusiones (.exe, *.out, .vscode/)
├── LICENSE                     <-- Licencia MIT escolar
├── README.md                   <-- Portada del proyecto
├── docs/                       <-- Carpeta de Documentación Formal
│   ├── InformeEEST1_LPR2026_ACT08_G03_v1.0.0.pdf <-- Informe en formato APA v7
│   ├── CHANGELOG.md            <-- Historial de versiones
│   └── manuales/
│       ├── manual_programador_v1.0.0.pdf <-- Manual Técnico (Ver plantilla)
│       └── manual_usuario_v1.0.0.pdf     <-- Guía de uso (Ver plantilla)
├── src/                        <-- Código fuente C++
│   └── main.cpp
└── capturas/                   <-- Evidencias de ejecución (.png)
    └── combate_consola.png     <-- Captura de tu consola con TU nombre y apellido
```

Resolución del Cuestionario Técnico de Control

1. **Diferencia técnica entre `struct` y `class`:** En C++, la única diferencia radica en la visibilidad por defecto que el compilador asigna. En un `struct`, todos los miembros son de acceso público (`public`) por defecto. En cambio, en una `class`, todos los miembros nacen protegidos bajo una frontera estrictamente privada (`private`), forzando las buenas prácticas de encapsulamiento.
2. **Peligro de dejar `vidas` en la sección pública:** Al exponer una variable crítica a la zona pública, cualquier parte externa del programa (o un bug en el ciclo de juego) podría reescribir su valor en la RAM de forma directa y descontrolada (ej: `jugador.vidas = -50`). Al colocarla en `private`, el objeto es el único capaz de alterar su salud de forma segura mediante un método controlado como `reducirVida()`.
3. **El rol del Constructor:** El constructor garantiza la integridad del objeto en el momento de su nacimiento. Su rol es reservar los bytes en la RAM e inicializar los atributos con valores válidos (como situar las coordenadas de la nave en el centro de la pantalla), evitando que el objeto inicie su ciclo con datos "basura" remanentes en la memoria.

4. **Ventaja de usar herencia para múltiples enemigos:** Si se requiriera agregar diferentes tipos de amenazas (naves exploradoras, cazas pesados, etc.), la herencia permitiría crear una clase base llamada `Enemigo` con las propiedades comunes (coordenadas `x`, `y` y `velocidad`). Las clases hijas heredarían este código automáticamente, y solo se tendría que programar la diferencia visual o su patrón de disparo, optimizando el desarrollo.

Conclusión

En esta sección se resumen las principales ideas trabajadas y se expresan opiniones personales sobre lo aprendido en la actividad. La migración hacia el paradigma de la Programación Orientada a Objetos brinda una estructura mucho más robusta y escalable al software en comparación con el diseño estructurado tradicional. Aprender a aislar los datos sensibles mediante encapsulamiento y modularizar las acciones a través de métodos de clase dota al desarrollador de competencias arquitectónicas críticas, fundamentales para diseñar aplicaciones eficientes y profesionales en los futuros desafíos de la materia.

Referencias

- Microsoft Learn. (2026). *Clases y estructuras en C++ - Encapsulamiento*. Microsoft Docs.
- Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-We

Link al Repositorio:

https://github.com/GabZac/PROGRAMACI-N-ORIENTADA-A-OBJETOS-POO-EN-C-JUEGO-DE-NAVES-EN-CONSOLA-class-Nave-.git